

MLQ | MLFQ

1. Multi-Level Queue Scheduling (MLQ)

- The ready queue is divided into **multiple separate queues based on priority**.
- A process is **permanently placed** into one queue depending on its nature (process type, memory use, priority, size).
(Once placed, it cannot move → rigid system)
- **Every queue can have its own scheduling policy**, such as:
 - System Processes → Round Robin
 - Interactive Processes → Round Robin
 - Batch Processes → FCFS
- **System processes** have the highest priority → created by OS.
Interactive processes require user input (foreground).
Batch processes run in the background without user interaction.
- Between these queues, scheduling is done using **fixed priority preemptive scheduling**.
Example: Foreground queue always runs before the background queue.
- If a new interactive process arrives while a batch process is executing, the batch process is **immediately preempted**.

Drawback:

- Lower-priority queues get CPU **only after higher-level queues finish**, which leads to **starvation**.
- **Convoy effect** is also seen in this method.

Multi-Level Queue Scheduling (MLQ) Examples

Remember → Process fixed in one queue forever (no movement).

Example 1 – School System

Queue	Students	Priority
Q1	Top rankers	Highest
Q2	Average students	Medium
Q3	Weak students	Lowest

- A top ranker always stays in Q1.
- Average student always remains in Q2.
- A weak student can never move to the top queue.

- So fixed classification → just like MLQ.

Example 2 – Restaurant Dining System

Queue	Eating Style	Priority
Q1	VIP dining area	Highest
Q2	Normal family tables	Medium
Q3	Outdoor economy seating	Lowest

- If a VIP comes, they are always served first.
- A normal customer cannot be moved to VIP section.
- No switching → fixed class based service.

Example 3 – Email Filtering

Queue	Category	Priority
Q1	Important mails	High
Q2	Social updates	Medium
Q3	Promotions/ads	Low

- Promotion email will never move to Important.
- Priority fixed just like MLQ permanent queue placement.

2. Multi-Level Feedback Queue Scheduling (MLFQ)

- Here also many queues exist, but with a key difference — **movement between queues is allowed**.
- A process is promoted or demoted based on how much CPU it uses:
 - If a process uses too much CPU time → moved to **lower priority queue**.
 - I/O-bound and interactive processes finish early → remain in **top priority queues**.
- If a process waits too long in a lower queue, it can be **moved upward (aging)** to avoid starvation.
- Starvation is **much less compared to MLQ**, because aging helps long-waiting jobs.
- More **flexible** than MLQ.
- Can be configured as needed for different types of operating systems.

Multi-Level Feedback Queue Scheduling (MLFQ) Examples

Remember → Processes can move up and down based on behavior.

Example 1 – Customer Care Call Center

Queue	Type	Priority
Q1	VIP callers	High
Q2	Normal callers	Medium
Q3	Long complaint / technical callers	Low

- Normal caller starts at Q2 → if issue quick, solved fast.
- If call takes long → moved to Q3.
- If someone is waiting too long in Q3 → pushed up again to Q2 (aging).

Example 2 – CPU Task Handling in Mobile

Queue	Task	Behavior
Q1	Notifications, alarms	High
Q2	Social media apps	Medium
Q3	Heavy apps/Games	Low

- New process starts in Q1 or Q2.
- Game using heavy CPU slowly moves to Q3.
- Notification comes → instantly interrupts and gets CPU.
- Long waiting app may shift up again (aging).

Example 3 – Library Book Issue System

Queue	Type of Request	Behavior
Q1	Quick return/renew	High
Q2	Issue new books	Medium
Q3	Bulk book issue for projects	Low

- If bulk request takes too long → handled later in Q3.
- Quick renewals get immediate service.
- If Q3 student is waiting too long → moved up temporarily to avoid starvation.

MLQ vs MLFQ

1. Core Difference

Feature	MLQ	MLFQ
Queue movement	No movement	Allowed (up & down)
Flexibility	Low	High
Starvation	High	Very low
Scheduling	Fixed priority	Dynamic priority
Used in	Simple static systems	Modern OS (Windows/Linux)

2. Advantages & Disadvantages

Approach	Advantages	Disadvantages
MLQ	Simple design, fast decision making	Starvation & Convoy effect, no flexibility
MLFQ	Less starvation, adapts to process behavior, faster response for interactive tasks	Implementation is complex, tuning queues & time slice is difficult

3.(Visual Memory Trick)

MLQ (Fixed)

+-----+ Highest Priority

| Queue-1 (SP) | ---> Only when empty, Queue-2 runs

+-----+

| Queue-2 (IP) |

+-----+

| Queue-3 (BP) | Lowest Priority

+-----+

MLFQ (Flexible)

+-----+ New process enters here

| Q1 (Highest) |

+-----↑-----↓-----+

| Q2 (Medium) | Uses more CPU → goes down

+-----↑-----↓-----+

| Q3 (Lowest) | Waits too long → moves up (aging)

+-----+